Real-Time HMM Map Matching: Replacing the SVM and the Gaussian Distance Feature

UE24CS352A Machine Learning mini-project | Anushka (PES1UG24CS074, Sec B) and Anushka P(PES1UG24CS075,Sec B

Code: https://github.com/codenoww/hmm-map-matching

1. Problem and approach

GPS readings are noisy, so a plotted drive often drifts off the road. **Map matching** snaps each reading to the road segment the vehicle was really on. We re-implement the HMM approach of the CS229 paper. The hidden state is the road segment and the observation is the GPS point. An **emission score** says how well a road fits one point (clues: distance to the road, agreement between the car's heading and the road direction, speed vs. speed limit). A **transition score** says how plausible a move from one road to the next is (clues: not turning back onto a recently visited road, and whether the distance between the two GPS points matches the distance between the two roads). The **Viterbi** algorithm picks the best whole chain of roads rather than the best road per point, so neighbouring points confirm each other and the path stops jumping. For real-time use it runs on a sliding window of T points and commits the decision for point i only after N more points have arrived (buffer $N \le T-1$); committed decisions are never changed.

The weight of each clue is **learned**, as in the paper (Sec. 4.3). On the training part of the drive, every possible move "road a at point i−1 to road b at point i" is one example with five clue values. The true move is labelled +1 and all others −1 (about 10% positive, so classes are balanced). An L1-penalised linear classifier is fitted, C is chosen by cross-validation, and its non-negative coefficients are normalised into the emission weights (3) and transition weights (2).

2. Our change (the delta)

We keep the paper's pipeline as the **baseline** (L1 SVM, Gaussian distance) and change two things, separately and together (a 2×2 experiment):

Change A, SVM → L1 logistic regression. Same features, same L1 penalty, class balancing and C search; only the classifier differs. Logistic regression is probabilistic, which fits an HMM whose scores are probabilities.

Change B, Gaussian → hyperbolic-secant distance feature. The paper scores a road by $\exp(-d^2/2\sigma^2)$; we use $1/\cosh(\pi d/2\sigma)$. Both peak at 1 when the road passes through the point, but sech has heavier tails, so a road that is somewhat far from a noisy point is penalised less harshly. σ is estimated from training points only (1.4826 × median distance to the true road).

3. Data and experimental setup

We use the public Newson & Krumm Seattle dataset (Microsoft Research): a 1 Hz GPS trace of 7,531 points, a road network, and a hand-verified ground-truth route. We cropped the network to roads near the drive (23,456 roads) and projected coordinates to metres (UTM zone 10N). For each point, roads within 50 m are candidates (about 4.5 per point; the true road is a candidate for 99.7% of points). To keep the comparison fair, the first 70% of the drive (in time order, no shuffling) is used for training and the last 30% (about 2,260 points) for testing, cross-validation uses contiguous blocks, and every variant sees the same split. Because our data and region differ from the paper's (Palo Alto), we compare only our variants against our own baseline and do not compare absolute numbers with the paper. Accuracy is the fraction of test points assigned to the correct road; error E = 1 − accuracy.

4. Results

Table 1. Test-drive results (vs baseline in percentage points; baseline highlighted).

Method	Distance feature	Accuracy	Error E	vs baseline
Nearest road (no HMM)	-	91.99%	8.01%	-3.01
HMM, hand-picked weights	Gaussian	95.04%	4.96%	+0.04
L1 SVM (paper baseline)	**Gaussian**	**95.00%**	**5.00%**	**0.00**
L1 SVM	Sech (B)	95.04%	4.96%	+0.04
L1 logistic regression (A)	Gaussian	94.95%	5.05%	-0.04
L1 logistic regression (A)	Sech (B)	95.09%	4.91%	+0.09

	Model	Distance feature	Accuracy (test drive)	Error E (test drive)	Change vs baseline (pct points)	best C	F1 (test pairs)	emission weights	transition weights
0	Nearest road (no HMM)	-	0.9199	0.0801	-3.0102	NaN	NaN	NaN	NaN
1	HMM, hand-picked weights	Gaussian (paper)	0.9504	0.0496	0.0443	NaN	NaN	NaN	NaN
2	L1 SVM (paper)	Gaussian (paper)	0.9500	0.0500	0.0000	1.0	0.4536	[0.45, 0.42, 0.14]	[0.66, 0.34]
3	L1 SVM (paper)	Hyperbolic secant (change B)	0.9504	0.0496	0.0443	1.0	0.4459	[0.44, 0.41, 0.15]	[0.67, 0.33]
4	L1 logistic regression (change A)	Gaussian (paper)	0.9495	0.0505	-0.0443	1.0	0.4551	[0.42, 0.44, 0.14]	[0.72, 0.28]
5	L1 logistic regression (change A)	Hyperbolic secant (change B)	0.9509	0.0491	0.0885	0.1	0.4481	[0.44, 0.42, 0.14]	[0.69, 0.31]

Figure A. Output of the experiment cell in our Colab run (the numbers in Table 1).

The HMM clearly helps. Every HMM variant reaches about 95% accuracy against 92% for simply picking the nearest road, which removes roughly 40% of the errors. **Our two changes do not matter much.** All four SVM/logistic-regression and Gaussian/sech combinations fall within 0.14 percentage points (about 3 test points) of each other, and the best, logistic regression with sech (95.09%), is only +0.09 points above the baseline. At this test size that difference is within noise, so we do not claim an improvement. The learned weights are also nearly identical across variants (emission about 0.44 / 0.42 / 0.14 for distance / direction / speed; transition 0.66–0.72 for the no-backtrack clue and 0.28–0.34 for the distance-travelled clue). The classifier F1 on held-out pairs is about 0.45 for every variant, which is modest because only about one in ten examples is positive. The practical reading is that this method is insensitive to the classifier and to the exact shape of the distance score, so logistic regression and sech are valid drop-in replacements.

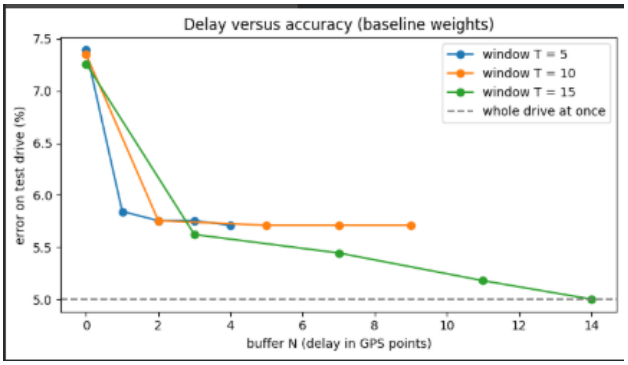


Figure 1. Delay vs accuracy (from our run). Error vs buffer N for T = 5, 10, 15. Deciding at once (N = 0) gives 7.3% error; waiting 3 points gives 5.6% and 14 points gives 5.0%, the full-drive level (dashed line). Most of the gain comes in the first few seconds.



Figure 2. Demo map (SR 520 / Overlake ramp, the 120-point stretch where the baseline makes the most mistakes). Green dashed = true route, blue = matched road, grey dots = correctly matched GPS points, red dots = mistakes.

5. Limitations and conclusion

We evaluated on a single 1 Hz drive from one city, with one train/test split, so the results are indicative rather than conclusive and cannot be compared directly with the paper's Palo Alto data. With a larger test set, or several drives, a small effect of the changes might become visible. In summary, we reproduced the paper's real-time HMM map matcher with learned weights, showed it cuts errors from 8.0% to about 5.0% over nearest-road matching, measured the delay-vs-accuracy trade-off of the sliding window, and found that replacing the SVM with logistic regression and the Gaussian with a hyperbolic-secant distance feature leaves accuracy unchanged within noise.